МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу “Компьютерная графика”

Текстурирование

Выполнил: Д. А. Кузнецов
Группа: М8О-307Б-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: Изучение принципов наложения текстур на 3D-объекты, загрузка растровых изображений из файловой системы, а также оптимизация работы с памятью GPU и передача данных через динамические uniform-буферы.

Выполненный вариант: Базовое задание. В рамках работы была внедрена поддержка текстур в качестве альбедо (основного цвета) для объекта из предыдущей лабораторной работы. Реализованы следующие задачи:

1. Загрузка изображения из файловой системы и декодирование его в формат RGBA с помощью сторонней библиотеки stb_image.
2. Добавление текстурных координат (UV) в структуру вершин и их процедурная генерация для граней параллелепипеда.
3. Отказ от использования Push Constants для матриц моделей и переход на один динамический Uniform Buffer (UBO).
4. Перемещение памяти для вершинных, индексных буферов и текстуры напрямую в видеопамять (VRAM) с использованием промежуточного (staging) буфера.

Метод решения

1. Загрузка текстур и UV-координаты. Для загрузки изображений из файла используется библиотека stb_image.h. Изображение загружается в оперативную память, после чего декодируется в 4 канала (RGBA). В структуру вершины (Vertex) вместо цвета было добавлено поле glm::vec2 texCoord. Поскольку геометрия генерируется процедурно, UV-координаты также задаются программно: для каждой из 6 граней создаются координаты от (0.0, 0.0) до (1.0, 1.0). В вершинном шейдере эти координаты передаются во фрагментный шейдер, где с помощью функции texture() происходит выборка цвета из sampler2D.

2. Динамический Uniform Buffer (UBO). В предыдущей работе уникальные данные каждого объекта (модельная матрица и цвет) передавались через Push Constants, что не позволяло передавать большие массивы данных. В данной работе было создано два UBO:

- Глобальный UBO (камера, свет, параметры материала), обновляемый раз в кадр.

- Динамический UBO (VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER_DYNAMIC), представляющий собой массив структур. При отрисовке каждого объекта (основного куба и кубика-источника света) в команде vkCmdBindDescriptorSets передается dynamicOffset, который указывает на нужную структуру в массиве. Это позволило использовать один дескрипторный набор для всех объектов на сцене.

3. Управление памятью GPU (VRAM) и Staging Buffers. Ранее вершинные и индексные данные лежали в оперативной памяти (RAM). Согласно заданию, они были перенесены в видеопамять (VRAM) с флагом VMA_MEMORY_USAGE_AUTO_PREFER_DEVICE. Так как GPU не может напрямую быстро читать данные из RAM, был реализован паттерн Staging Buffer. Данные (вершины, индексы и пиксели текстуры) сначала копируются во временный буфер в RAM (VK_BUFFER_USAGE_TRANSFER_SRC_BIT). Затем с помощью команд vkCmdCopyBuffer и vkCmdCopyBufferToImage данные переносятся в VRAM. После завершения копирования временный буфер уничтожается. Для текстуры также выполняется переход ее layout в VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL, чтобы GPU мог корректно читать ее из фрагментного шейдера.

Результаты

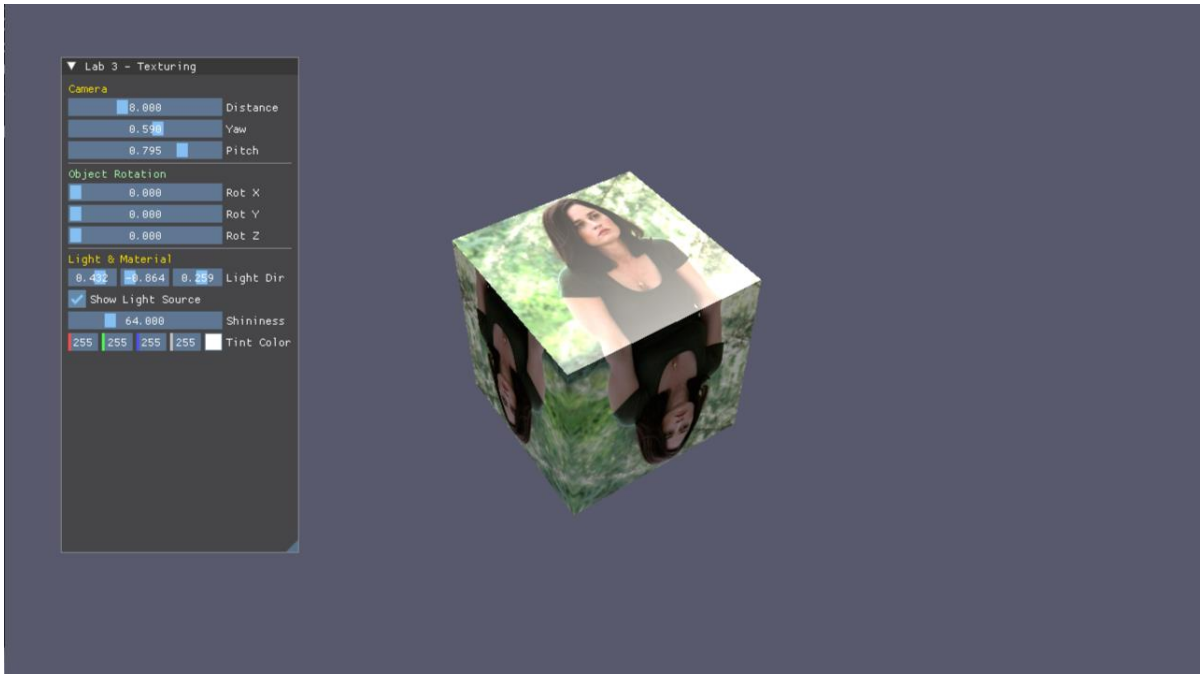


Рис. 1. Демонстрация наложения текстуры на грани объекта и корректная работа модели освещения Блинн-Фонга.

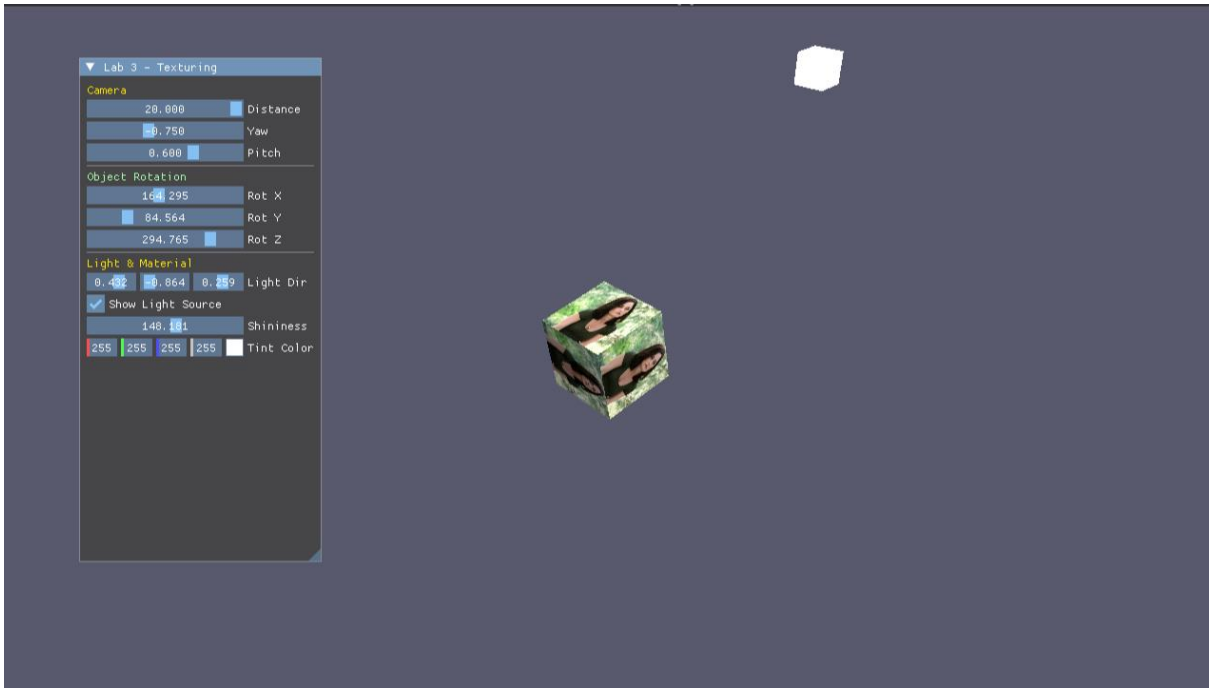


Рис. 2. Сцена с текстурированным параллелепипедом и визуализацией источника света (белый куб в правом верхнем углу).

Выводы

В ходе выполнения лабораторной работы были изучены принципы работы с текстурами в Vulkan API. Освоены методы загрузки изображений, создания VkImage и VkSampler. Также были углублены знания в области управления памятью GPU: реализовано использование staging-буферов для асинхронной и быстрой загрузки данных в VRAM. Переход на динамические Uniform Buffers позволил оптимизировать передачу свойств моделей и отказаться от ограничений Push Constants.